

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

print("TensorFlow:", tf.__version__)

texts = [
    "I am very happy today",
    "I feel amazing and wonderful",
    "Today is the best day",
    "I am so excited",
    "I got good news today",
    "I am feeling great",
    "I am really joyful",
    "I love this day",
    "I am extremely happy",
    "Everything is going perfectly",
    "I won the competition",
    "I received a wonderful gift",
    "My friends made me happy",
    "I am enjoying my day",
    "I feel fantastic",

    "I am feeling very sad",
    "Today was a terrible day",
    "I feel lonely",
    "I am disappointed",
    "I failed my exam",
    "I feel like crying",
    "I am unhappy today",
    "Nothing is going well",
    "I lost something important",
    "I miss my old friends",
    "I feel heartbroken",
    "I received bad news",
    "My result was disappointing",
    "I feel terrible",
    "I am having a bad day",

    "I am very angry",
    "This makes me furious",
    "I hate this situation",
    "I am extremely annoyed",
    "This is so irritating",
    "I am frustrated",
    "Stop bothering me",
    "I am angry about this",
    "This makes me mad",
    "I cannot tolerate this",
    "I am tired of this nonsense",
    "That behavior made me angry",
    "I am really irritated",
    "This is unacceptable",
    "I am losing my patience",

    "I went to college today",
    "The weather is normal",
    "I had lunch with my friend",
```

```
"I am going to class",
"Today is Monday",
"I bought a new book",
"The bus arrived on time",
"I completed my homework",
"I watched a movie",
"I am sitting in my room",
"I went to the library",
"I attended my class",
"I had breakfast",
"I travelled by bus",
"I opened my laptop"
]

labels = (
    ["Happy"] * 15 +
    ["Sad"] * 15 +
    ["Angry"] * 15 +
    ["Neutral"] * 15
)

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(labels)

tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)

X = pad_sequences(
    sequences,
    maxlen=20,
    padding="post"
)

X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.20,
    random_state=42,
    stratify=y_train
)

vocab_size = min(1000, len(tokenizer.word_index) + 1)

model = Sequential([
    Embedding(
        input_dim=vocab_size,
        output_dim=64,
        input_length=20
    ),
    LSTM(64),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(4, activation="softmax")
])
```

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

print("\nTraining model...\n")

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=30,
    batch_size=4,
    verbose=1
)

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\n=====")
print("MODEL EVALUATION")
print("=====")
print("Test Accuracy:", round(accuracy * 100, 2), "%")
print("Test Loss:", round(loss, 4))

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

def predict_emotion(sentence):

    sequence = tokenizer.texts_to_sequences([sentence])

    padded = pad_sequences(
        sequence,
        maxlen=20,
        padding="post"
    )

    prediction = model.predict(
        padded,
        verbose=0
    )[0]

```

```

index = np.argmax(prediction)

emotion = label_encoder.inverse_transform([index])[0]

confidence = prediction[index] * 100

emoji = {
    "Happy": "😊",
    "Sad": "😞",
    "Angry": "😡",
    "Neutral": "😐"
}

print("\n=====")
print("AI MOOD PREDICTION")
print("=====")
print("Input:", sentence)
print("Predicted Emotion:", emotion)
print("Emoji:", emoji[emotion])
print("Confidence:", round(confidence, 2), "%")

print("\nProbabilities:")

for name, probability in zip(
    label_encoder.classes_,
    prediction
):
    print(
        name + ":",
        round(probability * 100, 2),
        "%"
    )

print("\n=====")
print("MODEL READY")
print("=====")

predict_emotion(
    "I got selected for my dream job"
)

predict_emotion(
    "I failed my final examination"
)

predict_emotion(
    "This situation makes me extremely angry"
)

predict_emotion(
    "I went to the library today"
)

while True:

    sentence = input(
        "\nEnter a sentence (type exit to stop): "
    )

    if sentence.lower().strip() == "exit":
        print("Program ended.")
        break

    if sentence.strip():
        predict_emotion(sentence)

```


TensorFlow: 2.21.0
 /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument warnings.warn(
 Model: "sequential_5"

Layer (type)	Output Shape	Param #
embedding_5 (Embedding)	?	0 (unbuilt)
lstm_5 (LSTM)	?	0 (unbuilt)
dropout_10 (Dropout)	?	0
dense_10 (Dense)	?	0 (unbuilt)
dropout_11 (Dropout)	?	0
dense_11 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)

Training model...

Epoch 1/30
 10/10 ————— 4s 50ms/step - accuracy: 0.2632 - loss: 1.3935 - val_accuracy: 0.2000 -
 Epoch 2/30
 10/10 ————— 0s 18ms/step - accuracy: 0.2632 - loss: 1.3958 - val_accuracy: 0.2000 -
 Epoch 3/30
 10/10 ————— 0s 20ms/step - accuracy: 0.2895 - loss: 1.3892 - val_accuracy: 0.2000 -
 Epoch 4/30
 10/10 ————— 0s 21ms/step - accuracy: 0.2895 - loss: 1.3878 - val_accuracy: 0.2000 -
 Epoch 5/30
 10/10 ————— 0s 18ms/step - accuracy: 0.2632 - loss: 1.3898 - val_accuracy: 0.2000 -
 Epoch 6/30
 10/10 ————— 0s 17ms/step - accuracy: 0.2368 - loss: 1.3886 - val_accuracy: 0.2000 -
 Epoch 7/30
 10/10 ————— 0s 18ms/step - accuracy: 0.2895 - loss: 1.3854 - val_accuracy: 0.2000 -
 Epoch 8/30
 10/10 ————— 0s 18ms/step - accuracy: 0.2368 - loss: 1.3878 - val_accuracy: 0.2000 -
 Epoch 9/30
 10/10 ————— 0s 18ms/step - accuracy: 0.2368 - loss: 1.3917 - val_accuracy: 0.2000 -
 Epoch 10/30
 10/10 ————— 0s 18ms/step - accuracy: 0.1316 - loss: 1.3957 - val_accuracy: 0.2000 -
 Epoch 11/30
 10/10 ————— 0s 17ms/step - accuracy: 0.1842 - loss: 1.3840 - val_accuracy: 0.2000 -
 Epoch 12/30
 10/10 ————— 0s 19ms/step - accuracy: 0.3158 - loss: 1.3800 - val_accuracy: 0.3000 -
 Epoch 13/30
 10/10 ————— 0s 21ms/step - accuracy: 0.3158 - loss: 1.3803 - val_accuracy: 0.2000 -
 Epoch 14/30
 10/10 ————— 0s 18ms/step - accuracy: 0.3158 - loss: 1.3629 - val_accuracy: 0.2000 -
 Epoch 15/30
 10/10 ————— 0s 19ms/step - accuracy: 0.2895 - loss: 1.3278 - val_accuracy: 0.3000 -
 Epoch 16/30
 10/10 ————— 0s 18ms/step - accuracy: 0.4737 - loss: 1.1024 - val_accuracy: 0.3000 -
 Epoch 17/30
 10/10 ————— 0s 19ms/step - accuracy: 0.4737 - loss: 0.9771 - val_accuracy: 0.5000 -
 Epoch 18/30
 10/10 ————— 0s 21ms/step - accuracy: 0.4737 - loss: 0.8851 - val_accuracy: 0.5000 -
 Epoch 19/30
 10/10 ————— 0s 18ms/step - accuracy: 0.5000 - loss: 0.8182 - val_accuracy: 0.5000 -
 Epoch 20/30
 10/10 ————— 0s 18ms/step - accuracy: 0.6579 - loss: 0.6918 - val_accuracy: 0.5000 -
 Epoch 21/30
 10/10 ————— 0s 18ms/step - accuracy: 0.5000 - loss: 0.7716 - val_accuracy: 0.4000 -
 Epoch 22/30
 10/10 ————— 0s 18ms/step - accuracy: 0.3947 - loss: 1.5900 - val_accuracy: 0.7000 -
 Epoch 23/30
 10/10 ————— 0s 20ms/step - accuracy: 0.5000 - loss: 0.9829 - val_accuracy: 0.3000 -
 Epoch 24/30
 10/10 ————— 0s 19ms/step - accuracy: 0.4474 - loss: 1.0618 - val_accuracy: 0.2000 -
 Epoch 25/30

```

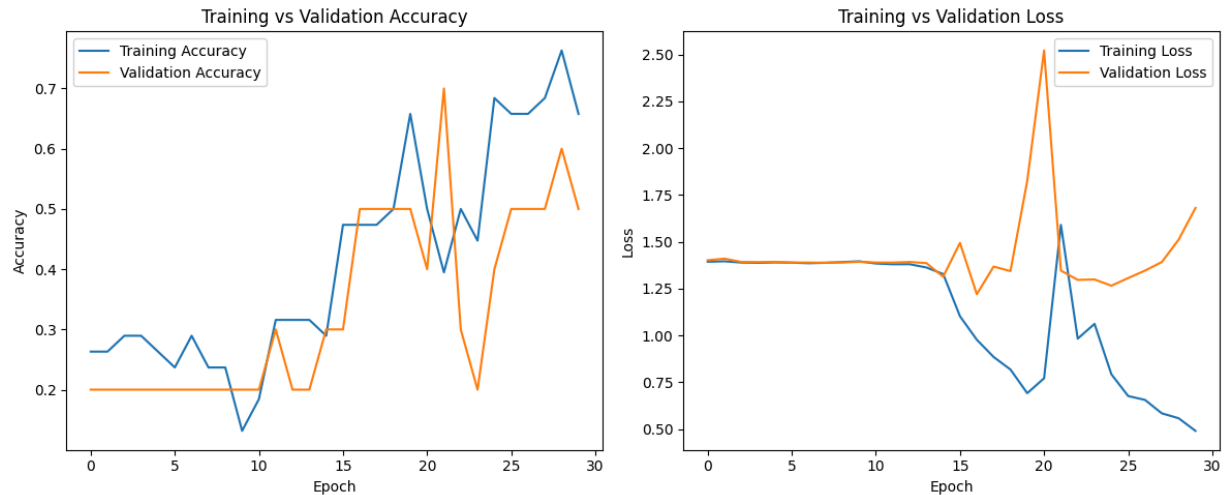
10/10 ----- 0s 19ms/step - accuracy: 0.6842 - loss: 0.7931 - val_accuracy: 0.4000
Epoch 26/30
10/10 ----- 0s 18ms/step - accuracy: 0.6579 - loss: 0.6765 - val_accuracy: 0.5000
Epoch 27/30
10/10 ----- 0s 18ms/step - accuracy: 0.6579 - loss: 0.6559 - val_accuracy: 0.5000
Epoch 28/30
10/10 ----- 0s 19ms/step - accuracy: 0.6842 - loss: 0.5843 - val_accuracy: 0.5000
Epoch 29/30
10/10 ----- 0s 22ms/step - accuracy: 0.7632 - loss: 0.5580 - val_accuracy: 0.6000
Epoch 30/30
10/10 ----- 0s 18ms/step - accuracy: 0.6579 - loss: 0.4907 - val_accuracy: 0.5000

```

```

=====
MODEL EVALUATION
=====
Test Accuracy: 25.0 %
Test Loss: 2.2111

```



```

=====
MODEL READY
=====

```

AI MOOD PREDICTION

```

=====
Input: I got selected for my dream job
Predicted Emotion: Sad
Emoji: 😞
Confidence: 52.97 %

```

Probabilities:

```

Angry: 0.57 %
Happy: 44.51 %
Neutral: 1.95 %
Sad: 52.97 %

```

AI MOOD PREDICTION

```

=====
Input: I failed my final examination
Predicted Emotion: Neutral
Emoji: 😐
Confidence: 82.79 %

```

Probabilities: